

MEDIATEK

MT8676 Android Camera Sensor Driver JSON Arch User Manual

版本: 1.0
出版日期: 2025-06-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2025-06-12	周健民	正式版

目录

版本记录 2

目录 3

1 Camera Driver JSON Arch 4

 1.1 概述 4

 1.2 配置/客制化指南 4

 1.2.1 如何添加一个新 Sensor 4

 1.2.2 如何添加一个新的 Tuning 文件 27

附件一 附加条款 29

1 Camera Driver JSON Arch

1.1 概述

本章节主要说明 MT8676 Camera 在解耦 Refactor 之后的 sensor 移植方法以及相关问题的调试思路及方法。

关于 Camera Refactor 的设计架构和原理，请参考如下文档：

- MT8678_JSON_Design_CN_V1.0.pptx

1.2 配置/客制化指南

本节主要介绍 Camera Refactor 解耦架构方式下的移植指南。

1.2.1 如何添加一个新 Sensor

JSON 化仅支持 YUV sensor

1.2.1.1 移植文件列表

Android:

- **Kernel Space**
 - \kernel\kernel_device_modules-6.1\arch\arm64\boot\dts\mediatek\cust_mt8676_camera_v4l2.dtsi
 - \vendor\mediatek\kernel_modules\mtkcam\imgsensor\src-v4l2\\$(sensor_name).json
 - \vendor\mediatek\kernel_modules\mtkcam\imgsensor\src-v4l2\sensor_bin_list.json
- **User Space**
 - \vendor\mediatek\proprietary\hardware\mtkcam-core\external\firmware\sensor\\$(sensor_name).bin
 - \vendor\mediatek\proprietary\hardware\mtkcam-core\external\firmware\sensor\sensor_bin_list.bin
 - \vendor\mediatek\proprietary\hardware\mtkcam-core\external\firmware\camera\metadata\\$(sensor_name)\\$(sensor_name).json
 - \vendor\mediatek\proprietary\hardware\mtkcam-core\external\firmware\camera\tuning\\$(sensor_name)_tuning_param.json

1.2.1.2 在 Kernel 中添加 Sensor Firmware 文件

Android:

- **Kernel Space**
 - \vendor\mediatek\kernel_modules\mtkcam\imgsensor\src-v4l2

• User Space

- \vendor\mediatek\proprietary\hardware\mtkcam-core\external\firmware\sensor\\$(sensor_name).bin
- \vendor\mediatek\proprietary\hardware\mtkcam-core\external\firmware\sensor\sensor_bin_list.bin

\$(sensor_name).json 文件由专用工具：**Camera Firmware JSON Generator** 产生。此工具是一个 web 网页界面，开发者可根据页面提示，填入表单所需的内容，最后点击保存即可生成\$(sensor_name).json 文件。

网页工具表单的参数内容含义，可参考 *MT8678 JSON Design CN.ppt* 文档介绍。

Camera Firmware Json Generator - For Automotive
MEDIATEK

Form

```

=== camera_module ===
* name:
YourSensor_mipi_yuv

* compatible_dts_mask:
0x1

* module_id:
0x52

* module_id_is_unique:
YES

i2c_addr_table
0x52
0xff
          
```

Json

```

1 {
2   "camera_module": {
3     "name": "YourSensor_mipi_yuv",
4     "compatible_dts_mask": "0x1",
5     "module_id": "0x52",
6     "module_id_is_unique": 1,
7     "module_id_virtual": "0xff",
8     "i2c_addr_table": [
9       "0x52",
10      "0xff",
11      "0xff",
12      "0xff",
13      "0xff"
14    ],
15     "reg_addr_sensor_id": [
16       "0x00",
17       "0x00",
18       "0x00"
19    ],
20     "i2c_transfer_data_type": 0,
21     "reg_cnt_for_id": 1
22   },
23   "serdes": {
24     "serializer": {
25       "vendor": "maxim",
26       "name": "max92xxx",
27       "index": 0,
28       "linked_deser_idx": 0,
29       "i2c_transfer_data_type": 0,
30       "vc_id": 0
31     }
32   }
33 }
          
```

生成的\$(sensor_name).json 文件需要保存到\vendor\mediatek\kernel_modules\mtkcam\imgsensor\src-v4l2\目录。

j1_00 > merged > vendor > mediatek > kernel_modules > mtkcam > imgsensor > src-v4l2 >

Name	Date modified	Type	Size
sensor_bin_list	4/28/2025 7:08 PM	JSON File	1 KB
automotive-entry	4/22/2025 6:02 PM	C Source	56 KB
adaptor-ctrls	3/31/2025 8:06 PM	C Source	81 KB
adaptor-i2c	3/31/2025 8:06 PM	C Source	11 KB
adaptor-i2c	3/31/2025 8:06 PM	H File	2 KB
adaptor-subdrv	3/31/2025 8:06 PM	H File	18 KB
adaptor-subdrv-ctrl	3/31/2025 8:06 PM	C Source	131 KB
automotive-entry	3/31/2025 8:06 PM	H File	8 KB
Kbuild	3/31/2025 8:06 PM	File	4 KB
max96712_mipi_yuv_ss220at	3/31/2025 8:06 PM	JSON File	33 KB
max96712a_mipi_yuv	3/31/2025 8:06 PM	JSON File	33 KB
max96712a0_mipi_yuv	3/31/2025 8:06 PM	JSON File	14 KB
max96712a1_mipi_yuv	3/31/2025 8:06 PM	JSON File	14 KB
max96712a2_mipi_yuv	3/31/2025 8:06 PM	JSON File	14 KB
max96712a3_mipi_yuv	3/31/2025 8:06 PM	JSON File	14 KB

在 src-v4l2/目录下，执行 `python3 gen_sensor_firmware.py ./sensor_name.json` 命令。

```
gen_sensor_firmware.py
```

```
./gen_sensor_firmware.py ./max96712_mipi_yuv_ss220at.json
```

脚本执行成功后，在 `src-v4l2/` 目录下生成 `sensor_name.bin` 文件。

在 `sensor_bin_list.json` 中添加上面生成的 `sensor_name.bin` 文件，并且生成 `sensor_bin_list.bin` 文件。

```
sensor_bin_list.json X
src-v4l2 > {} sensor_bin_listjson > ...
2  "sensor_bin_list": [
3    "max96712_mipi_yuv.bin",
4    "max96712a_mipi_yuv.bin",
5    "max96712a0_mipi_yuv.bin",
6    "max96712a1_mipi_yuv.bin",
7    "max96712a2_mipi_yuv.bin",
8    "max96712a3_mipi_yuv.bin"
9  ]
10
```

```
./gen_sensor_firmware.py ./sensor_bin_list.json
```

```
vendor > mediatek > kernel_modules > mtkcam > imgsensord > src-v4l2 >
```

Name	Date modified	Type	Size
 sensor_bin_list.bin	5/19/2025 5:31 PM	BIN File	1 KB
 max96712_mipi_yuv.bin	5/19/2025 5:25 PM	BIN File	3 KB

将生成的 `$(sensor_name).bin` 和 `sensor_bin_list.bin` 二进制文件，通过 `adb push` 到开发板的

`/vendor/firmware/sensor/` 目录，重启开发板即可。

```
_64_bsp:/vendor/firmware/sensor # ls -l
total 8
-rw-r--r-- 1 root root 1751 2023-12-28 05:40 sc20at_mipi_yuv.bin
-rw-r--r-- 1 root root 1751 2023-12-28 05:40 sc21at_mipi_yuv.bin
-rw-r--r-- 1 root root 1751 2023-12-28 05:40 sc22at_mipi_yuv.bin
-rw-r--r-- 1 root root 1751 2023-12-28 05:41 sc23at_mipi_yuv.bin
```

注：通过 `adb push firmware.bin` 文件到开发板的 `/vendor/firmware/sensor/` 目录的方式，可以方便固件调试，无需重复 `rebuild driver source code`。

在确认 `firmware` 无需继续修改后，保存好 `JSON file`，并将更新的 `.bin` 文件复制到

`\vendor\mediatek\proprietary\hardware\mtkcam-core\external\firmware\sensor\` 目录，下次 `full build`

代码即可将所有 `firmware` 文件编译到 `image` 中。整体烧录后，开发板系统即包含了所有的 `firmware.bin` 文件。

1.2.1.3 修改 dts 文件

Android:

`\kernel\kernel_device_modules-6.1\arch\arm64\boot\dts\mediatek\cust_mt8676_camera_v4l2.dtsi`

通过分析硬件原理图找到当前 sensor 是挂在哪个 I2C 下面，然后在对应的 I2C 下面增加 sensor 配置，如下所示：

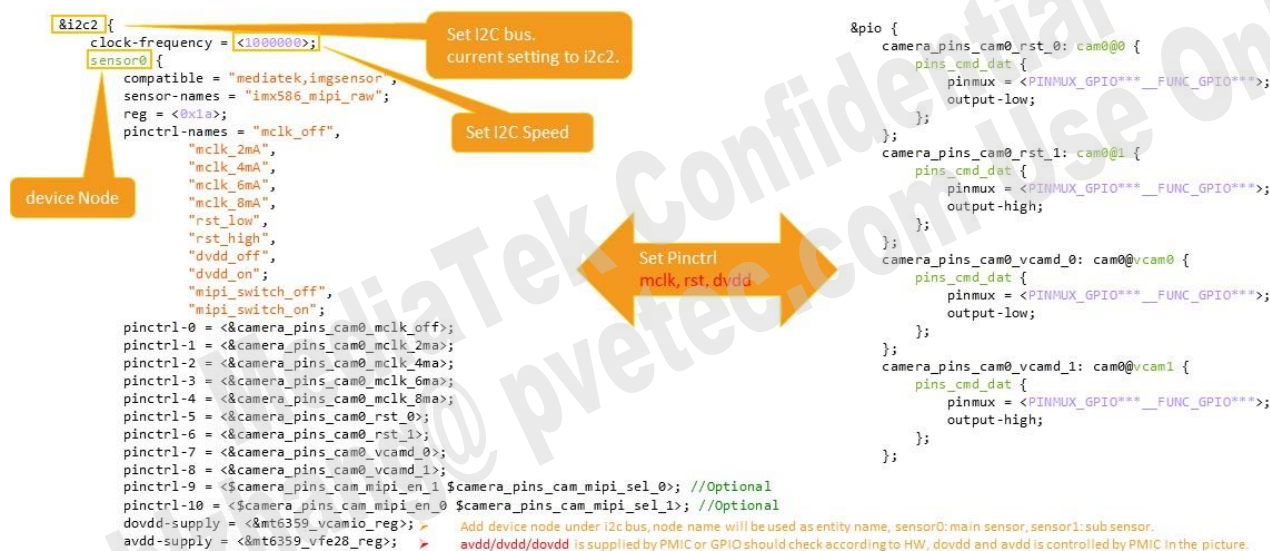
- 增加了 sensor0

注：如果该 sensor 需走 legacy 方式，则需要在 sensor node 下新增一个属性：**mediatek, legacy-search;**

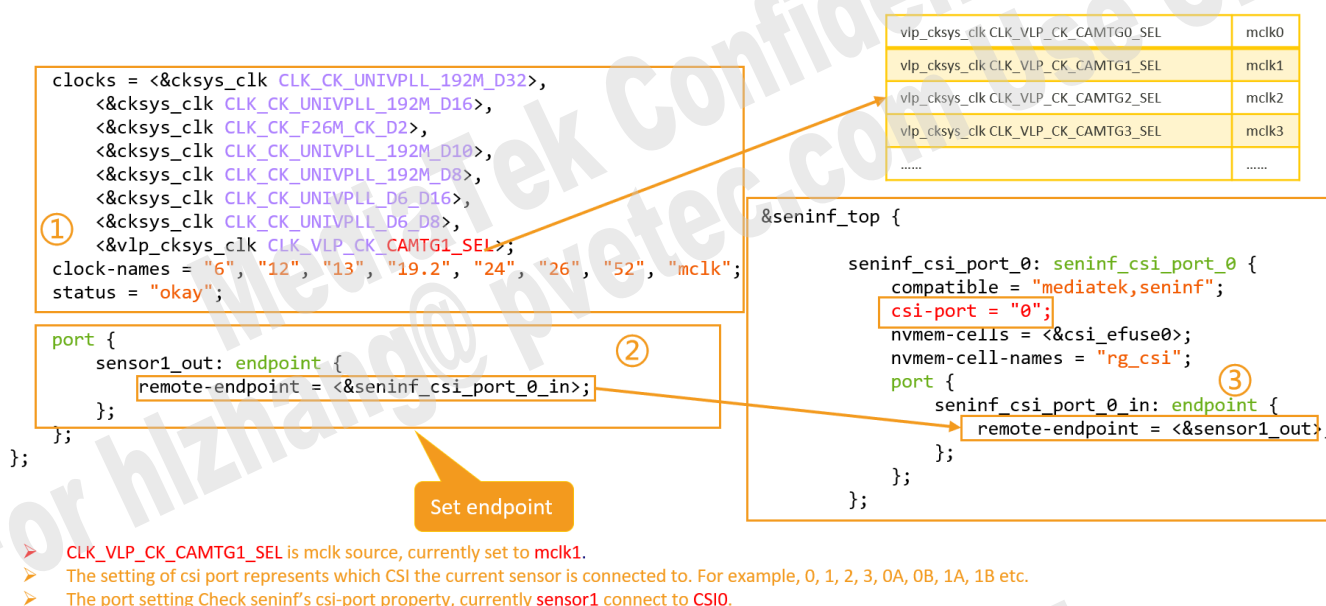
```
sensor1: sensor1@52 {
    » mediatek, legacy-search;
    » compatible = "mediatek, imgsensor";
    » sensor-names = "max96712_mipi_yuv";
    » reg = <0x52>;

    » #thermal-sensor-cells = <0>;
    » pdn-supply = <&gpio47_regulator>;

    » clocks = <&vlp_cksys_clk CLK_CK_UNIVPLL_192M_D32>,
    »         <&vlp_cksys_clk CLK_CK_UNIVPLL_192M_D16>,
    »         <&vlp_cksys_clk CLK_CK_F26M_CK_D2>,
    »
```



关于 `seninf` 和 `sensor` 的关联，可以参考下图，`seninf_top` 里 `csi-port` 的值是硬件实际连接的 `csi-port`。



与 dts 里配置对应的就是上电时序的部分（如下所示），需要参考 sensor 规格书中的上电时序来配置。

1. Add power sequence info

- {power_type, voltage, delay_time in ms}
- Power on sequence from top to bottom
- Power off sequence from bottom to top
- mclk set frequency(24M) and drive current(4mA)

2. Add subdrv entry

- Naming rule: \$(folder-name)_entry
- Macro will generate sensor-list automatically

- Rename a copy according to the existing sensor driver source code, and the part of the register and sensor info should be confirmed with the vendor.

```

common/imx586_mipi_raw/imx586mipiraw_Sensor.c

static struct subdrv_pw_seq_entry pw_seq[] = {
    {HW_ID_MCLK, 24, 0},
    {HW_ID_RST, 0, 0},
    {HW_ID_AVDD, 2800000, 0},
    {HW_ID_AFVDD, 2800000, 0},
    {HW_ID_DVDD, 1100000, 0},
    {HW_ID_DOVDD, 1800000, 1},
    {HW_ID_MCLK_DRIVING_CURRENT, 4, 1},
    {HW_ID_RST, 1, 3},
};

const struct subdrv_entry imx586_mipi_raw_entry = {
    .name = "imx586_mipi_raw",
    .id = IMX586_SENSOR_ID,
    .pw_seq = pw_seq,
    .pw_seq_cnt = ARRAY_SIZE(pw_seq),
    .ops = &ops,
};

```

在最新的 max96712mipiyuv_sensor.c 代码版本上 pw_seq 的格式如下：

```

969 static struct subdrv_pw_seq_entry pw_seq[] = {
970     {HW_ID_PDN, 0, 20},
971     {HW_ID_PDN, 1800000, 20},
972 };

```

1.2.1.4 增加新的 meta 文件

复制一份 metadata 模板文件夹（\vendor\mediatek\proprietary\hardware\mtkcam-core\external\firmware\metadata\max96712_mipi_yuv），将文件夹名称修改为当前移植的 sensor 名字（例如 xxxx_mipi_yuv）。

将 xxxx_mipi_yuv 文件夹下的所有 JSON 文件名中及对应 JSON 文件内的 sensor name，修改为当前移植的 sensor 的名字，注意字母的大小写格式要跟模板文件的写法一致。

以 max96712_mipi_yuv 为模板，需要将所有包含 max96712_mipi_yuv、MAX96712_MIPI_YUV 字样的 JSON 文件夹名称修改为当前所移植的 sensor 名字。

merged > src > multimedia > camera-hal > mt8676 > mtkcam-core > external > firmware > metadata > max96712_mipi_yuv

Name	Date modified	Type
constructCustRequestMetadata_DEVICE_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_COMMON_SENSOR_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_AVAILABLE_KEYS_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_CUSTOM_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_LENS_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_MOD_OVERWRITE_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_MODULE_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_PRO_OVERWRITE_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_PROJECT_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PLATFORM_VENDOR_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File
constructCustStaticMetadata_PROJECT_V1_PRO_OVERWRITE_SENSOR_DRVNAME_MAX96712_MIPI_YUV	4/7/2025 7:01 PM	JSON File

```

1  {
2      "function_name": "constructCustStaticMetadata_COMMON_SENSOR_SENSOR_DRVNAME_MAX96712_MIPI_YUV",
3      "metadata": [
4          {
5              "condition": "always",
6              "type": "CONFIG_METADATA",
7              "tag": "MTK_SENSOR_BLACK_LEVEL_PATTERN",
8              "values": [
9                  { "value": 0, "type": "MINT32" },
10                 { "value": 0, "type": "MINT32" },
11                 { "value": 0, "type": "MINT32" },
12                 { "value": 0, "type": "MINT32" }
13             ]
14         }
15     ]
16 }

```

注:

- 一颗 sensor 有好几份不同类型的 JSON 文件（都是以 _SENSOR_DRVNAME_XXX.json 作为后缀），每一份 JSON 都需要按照上面的方式修改。
- 如果是 raw 或者其他类型的 sensor，就要复制对应 sensor 类型的 JSON 并以此为基础进行修改。
- 如需修改分辨率，可以参考下面的 JSON，改为新的分辨率。

```

mtkcam-core > external > firmware > metadata > {} constructCustStaticMetadata PLATFORM_PROJECT SENSOR_DRVNAME_MAX96712_MIPI_YUV.json >
3      "metadata": [
17      ],
18      {
19          "condition": "always",
20          "type": "CONFIG_METADATA",
21          "tag": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_WITH_DURATIONS",
22          "values": [
23              {"value": 33, "type": "MINT64"},
24              {"value": 1280, "type": "MINT64"},
25              {"value": 720, "type": "MINT64"},
26              {"value": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT", "type": "MINT64"},
27              {"value": 33333333, "type": "MINT64"},
28              {"value": 0, "type": "MINT64"},
29              {"value": 32, "type": "MINT64"},
30              {"value": 1280, "type": "MINT64"},
31              {"value": 720, "type": "MINT64"},
32              {"value": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT", "type": "MINT64"},
33              {"value": 33333333, "type": "MINT64"},
34              {"value": 0, "type": "MINT64"},
35              {"value": 35, "type": "MINT64"},
36              {"value": 1280, "type": "MINT64"},
37              {"value": 720, "type": "MINT64"},
38              {"value": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT", "type": "MINT64"},
39              {"value": 33333333, "type": "MINT64"},
40              {"value": 0, "type": "MINT64"},
41              {"value": 35, "type": "MINT64"},
42              {"value": 960, "type": "MINT64"},
43              {"value": 540, "type": "MINT64"},
44              {"value": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT", "type": "MINT64"},
45              {"value": 33333333, "type": "MINT64"},
46              {"value": 0, "type": "MINT64"},
47              {"value": 35, "type": "MINT64"},
48              {"value": 640, "type": "MINT64"},
49              {"value": 480, "type": "MINT64"},
50              {"value": "MTK_SCALER_AVAILABLE_STREAM_CONFIGURATIONS_OUTPUT", "type": "MINT64"},
51              {"value": 33333333, "type": "MINT64"},
52              {"value": 0, "type": "MINT64"},
53          ]
54      },

```

调整分辨率的json文件

黄色线框出来的是分辨率，1280是width，720是height

修改完毕后，将 xxxx_mipi_yuv/ 文件夹，通过 adb push 的方式，push 到开发板的

/vendor/firmware/camera/metadata/ 文件夹，重新启动即可。

在确认 metadata 无需继续修改后，请将对应的 metadata 文件放到

\vendor\mediatek\proprietary\hardware\mtkcam-

core\external\firmware\metadata\max96712_mipi_yuv 目录下，然后执行完整编译工程的命令，即可将所有

firmware 文件编译到 image 中。整体烧录后，开发板系统即包含了所有的 metadata 文件。

注：若在 full build 过程中出现类似下面的错误，则表示不同版本之间会有些原生 basic tag 的差异，版本迁移的时候会同步更新一些 tag 到 mtk_metadata_tag.h，进而需要使用脚本更新 MapStringToEnum.cpp。

```

mtkcam-core/external/MapStringToEnum.cpp:738:41: error: use of undeclared identifier
'MTK_MFNR_FEA                                     TURE_STORE_BSS2META'; did you mean
'MTK_MFNR_FEATURE_START'?
{"MTK_MFNR_FEATURE_STORE_BSS2META", MTK_MFNR_FEATURE_STORE_BSS2META},
^~~~~~

```

更新脚本执行方法：进入 mtkcam-core/external/ 目录，执行 python MapStringToEnum.py。

再次重新进行 full build 命令即可。

1.2.1.5 配置 Sensor Format

请在 web 界面 **Camera Firmware JSON Generator** 中修改。

根据 bridge 输出的 format 配置 UYVY 或者 YUYV:

sensor_output_dataformat:

SENSOR_OUTPUT_FORMAT_UYVY

查找所有配置 sensor_output_dataformat 的地方改为实际的 format。

1.2.1.6 配置 modestruct

请在 web 界面 **Camera Firmware JSON Generator** 中修改。

与 vendor 确认 setting 的相关参数填入到结构体中。

Pclk->pixel clock, 单位 Hz

Linlength->HTS

Framelength->VTS

$\text{Linlength} \times \text{framelength} = \text{pclk}$

Grabwindow_width-> camera size 宽 (在 MT8676 上没有这个值, 之后会填在 solution 的位置)

Grabwindow_height-> camera size 高 (在 MT8676 上没有这个值, 之后会填在 solution 的位置)

Mipi_pixel_rate-> $\text{mipi datarate} \times \text{lane_number/bitdepth}$, YUV bitdepth = 16



The screenshot shows the 'mode_struct' configuration section of the Camera Firmware JSON Generator. The fields are as follows:

- hdr_mode: 0
- raw_cnt: 1
- exp_cnt: 1
- pclk: (empty field)
- linlength: (empty field)
- framelength: (empty field)

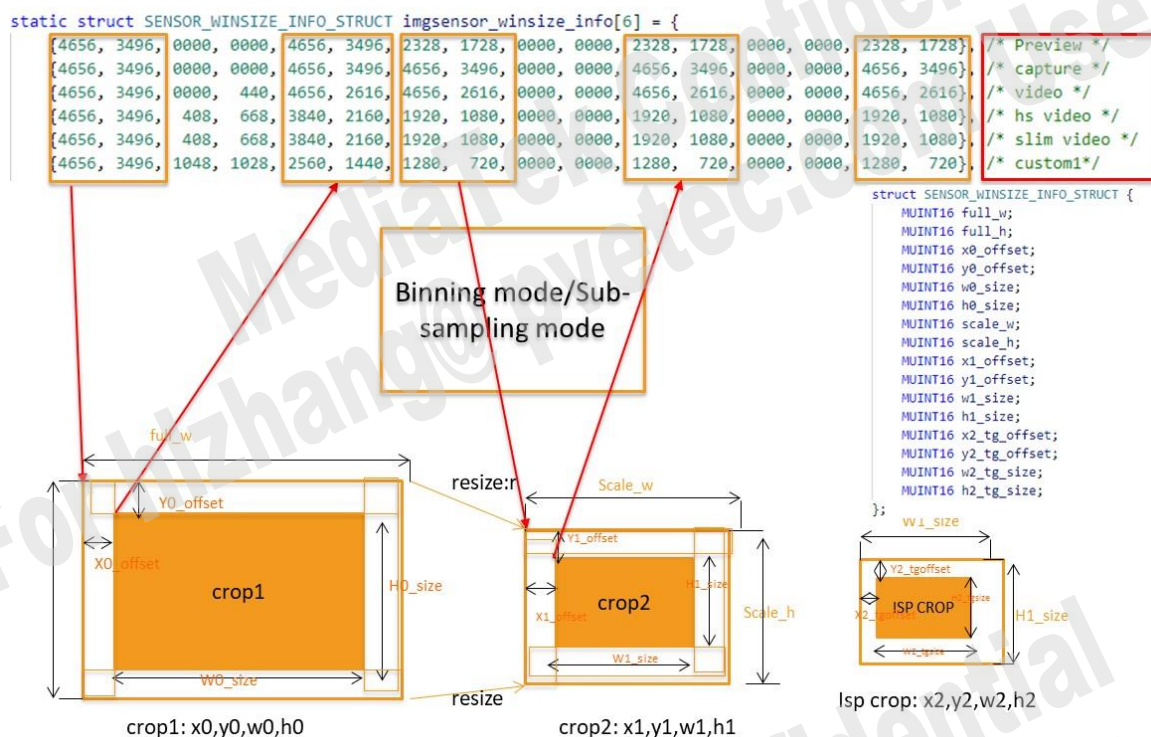
The corresponding JSON output for the mode_struct is shown on the right:

```

51 },
52 "frame_desc": [
53 ],
54 "mode_struct": [
55 {
56   "hdr_mode": 0,
57   "raw_cnt": 1,
58   "exp_cnt": 1,
59   "pclk": undefined,
60   "linlength": undefined,
61   "framelength": undefined,
62   "max_framerate": undefined,
63   "mipi_pixel_rate": undefined,
64   "readout_length": 0,
65   "read_margin": 10,
66   "framelength_step": 2,
67   "coarse_integ_step": 2,
68   "pdaf_cap": 1,
69   "ae_binning_ratio": 1465,
70   "fine_integ_line": 826,
71   "delay_frame": 3,
72   "dpc_enabled": 1,
73   "csi_param": {
74     "dphy_trail": undefined,
75     "dphy_data_settle": undefined,
76     "dphy_clk_settle": undefined,
77     "cphy_settle": undefined,

```

下图是借用了之前版本的一个对 winsize info 的解释, 来描述各个字段的含义:



限制: full size (4:3) 或 full size (16:9) 的宽和高需要进行 4 倍数对齐。

配置 sensor 输出的 size 宽高, 与 Grabwindow_width/Grabwindow_height size 一致

Grab window 要求:

1. 设置的宽必须是 16 的倍数, 高必须为 4 的倍数。
2. 相同比例下视角一致: 4:3 与 16:9 要求 sensor 输出的 window 保证水平方向和 full size 视角一致。
3. Grab window 设置建议和 sensor 输出的 window 一致。

特殊情况下是: $\text{grabwindow_width}(\text{height}) \leq \text{sensor output width}(\text{height}) - \text{startx}(\text{y})$

1.2.1.7 配置 static_ctx

请在 web 界面 Camera Firmware JSON Generator 中修改。

如下图所示:

1. sensor_id 字段是填对应的 sensor ID。
2. i2c_add_table 一般只配置 0x52 即可, 就是实际的解串器的地址。
3. 配置分辨率, 这里就是前面说的 grabwindow 的值。
4. 配置 mipi_lane_num, 这里表示是 MIPI 的 lane 的个数。

subdrv_static_ctx

* i2c_burst_write_support:

1

* i2c_transfer_data_type:

I2C_OT_ADDR_16_DATA_8

resolution

[width, height]

1920

1080

```

111 "subdrv_static_ctx": {
112   "i2c_burst_write_support": 1,
113   "i2c_transfer_data_type": 0,
114   "resolution": [
115     1920,
116     1080
117   ],
118   "mirror": 3,
119   "mclk": 24,
120   "isp_driving_current": 1,
121   "sensor_interface_type": 1,
122   "mipi_sensor_type": 0,
123   "mipi_lane_num": {
124     "value": 0
125   },
126   "sensor_output_dataformat": 4,
127   "frame_length_max": 65532,
128   "ae_effective_frame": 2,
129   "frame_time_delay_frame": 3,
130   "start_exposure_offset": 500000,
131   "seamless_switch_support": 0,
132   "temperature_support": 0,
133   "reg_addr_stream": 256,
134   "reg_addr_mirror_flip": 257,
135   "chk_s_off_sta": 1,
136   "chk_s_off_end": 0
137 }

```

5. `cam_type` 是按照模组场景的类型填写，如果是单 sensor 的话，可以填 `SENSOR_TYPE_SINGLE`，它有如下值可以选择：

```

523 enum SPM_CAM_TYPE_ENUM {
524     SENSOR_TYPE_SINGLE = 0,
525     SENSOR_TYPE_COMB = 1,
526     SENSOR_TYPE_MULT = 2,
527     SENSOR_TYPE_NON_COMB = 3,
528     SENSOR_TYPE_COMB_AVM = 4,
529     SENSOR_TYPE_NON_COMB_AVM = 5,
530     SENSOR_TYPE_MULT_ASYNC = 6,
531     SENSOR_TYPE_EXTCAM = 7,
532     SENSOR_TYPE_NONE
533 };

```

- `SENSOR_TYPE_COMB_AVM` 表示组帧 AVM
- `SENSOR_TYPE_SINGLE` 表示单路 VC sensor
- `SENSOR_TYPE_MUTI` 表示 multicam
- `SENSOR_TYPE_MULT_ASYNC` 表示各路 VC 相互独立不影响的 multicam
- `SENSOR_TYPE_NON_COMB_AVM` 表示非组帧 AVM

6. `sensor_output_dataformat` 表示模组输出的数据的 format，填实际的 format 即可。

`sensor_output_dataformat`:

SENSOR_OUTPUT_FORMAT_UYVY

SENSOR_OUTPUT_FORMAT_UYVY

SENSOR_OUTPUT_FORMAT_VYUY

SENSOR_OUTPUT_FORMAT_YUYV

SENSOR_OUTPUT_FORMAT_VYVU

7. 关于 multicam group info 的配置，在 JSON web tool 里可以参考如下配置：

最终生成到 JSON file 里会是下面这样的格式，分别是 1 个 master sensor 和 4 个 slave sensor 的 dts 里的 sensor ID 和 VC 信息。

```

959     "multicam_group_inf": {
960         "master_dts_idx": 11,
961         "vc_num": 4,
962         "slave_group_info": [
963             {
964                 "slave_dts_idx": 7,
965                 "vc_id": 0,
966                 "link_status": 0
967             },
968             {
969                 "slave_dts_idx": 8,
970                 "vc_id": 1,
971                 "link_status": 0
972             },
973             {
974                 "slave_dts_idx": 9,
975                 "vc_id": 2,
976                 "link_status": 0
977             },
978             {
979                 "slave_dts_idx": 10,
980                 "vc_id": 3,
981                 "link_status": 0
982             }
983         ]
984     },
985 },

```

如果配置为 SENSOR_TYPE_MULTI_ASYNC，还需要添加如下配置：

```

1116     "multicam_group_dt_inf": {
1117         "vc_num": 4,
1118         "sensor_group_dt_info": [
1119             {
1120                 "vc_id": 0,
1121                 "vc_data_type": 30,
1122             },
1123             {
1124                 "vc_id": 1,
1125                 "vc_data_type": 30
1126             },
1127             {
1128                 "vc_id": 2,
1129                 "vc_data_type": 30
1130             },
1131             {
1132                 "vc_id": 3,
1133                 "vc_data_type": 30
1134             }
1135         ]
1136     },
1137 },

```

vc num 4
VC channel 0
data_type yuv422

1.2.1.8 配置 VC 信息

请在 web 界面 **Camera Firmware JSON Generator** 中修改。

如果是单个 VC，可以进行如下配置：

frame_desc

* channel:

0

* data_type:

0

* dt_remap_to_type:

0

```

51 },
52 "frame_desc": [
53 {
54   "channel": 0,
55   "data_type": 0,
56   "dt_remap_to_type": 0,
57   "hsize": 1,
58   "vsize": 1,
59   "user_data_desc": 0,
60   "fs_seq": 0
61 }
62 ],
63 "mode_struct": [
64 {
65   "hdr_mode": 0,
66   "raw_cnt": 1,
67   "exp_cnt": 1,

```

如果是多个 VC，主要是修改 channel 和 user_data_desc。

1.2.1.9 配置获取 Sensor ID

请在 web 界面 Camera Firmware JSON Generator 中修改。

=== camera_module ===

* name:

YourSensor_mipi_yuv

* compatible_dts_mask:

0x1

* module_id:

0x52

* module_id_is_unique:

YES

```

1 {
2   "camera_module": {
3     "name": "YourSensor_mipi_yuv",
4     "compatible_dts_mask": "0x1",
5     "module_id": "0x52",
6     "module_id_is_unique": 1,
7     "module_id_virtual": "0xff",
8     "i2c_addr_table": [
9       "0x52",
10      "0xff",
11      "0xff",
12      "0xff",
13      "0xff"
14    ],
15     "reg_addr_sensor_id": [
16       "0x00",
17       "0x00",
18       "0x00"
19    ],

```

1.2.1.10 配置 Sensor Init

请在 web 界面 Camera Firmware JSON Generator 中修改。

主要是配置模組的设置，需要客户和 vendor 调试生成

=== reg_init_settings ===

init_serdes_reg_info

+

init_cmos_reg_info

+

```

38   }
39 },
40 "reg_init_settings": {
41   "init_serdes_reg_info": [
42   ],
43   "init_cmos_reg_info": [
44   ]
45 },
46 "reg_streaming_settings": {
47   "stream_on_reg_info": [
48   ],
49   "stream_off_reg_info": [
50   ]
51 },
52 "frame_desc": [
53 {
54   "channel": 0,

```

1.2.1.11 配置 get_sensor_usage

请在 web 界面 **Camera Firmware JSON Generator** 中修改。

subdrv_entry

* sensor_usage:

MTK_SENSOR_USAGE_SINGLE

163

164

165

166

167

168

},

"pw_seq": [

],

"subdrv_entry": {

"sensor_usage": 0

},

它的值可以选择下面几种，COMB 表示组帧 AVM，MUTI 表示 multicam，NONCOMB 表示非组帧 AVM，MUTICAM_ASYNC 表示 slave 独立不会相互影响的 multicam，单 sensor 的话配置为 SINGLE。

```
42 enum mtk_sensor_usage {
43     MTK_SENSOR_USAGE_SINGLE,
44     MTK_SENSOR_USAGE_COMB,
45     MTK_SENSOR_USAGE_MUTI,
46     MTK_SENSOR_USAGE_NONCOMB,
47     MTK_SENSOR_USAGE_MUTI_ASYNC,
48 };
```

1.2.1.12 配置 Streaming Control

请在 web 界面 **Camera Firmware JSON Generator** 中修改。

初始化的设定全部放到 `sensor_init()` 函数里。MIPI enable 和 disable 函数放到 `streaming_control()` 函数中。

=== reg_streaming_settings ===

stream_on_reg_info

+

stream_off_reg_info

+

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

"init_cmos_reg_info": [

],

"reg_streaming_settings": {

"stream_on_reg_info": [

],

"stream_off_reg_info": [

]

},

"frame_desc": [

{

"channel": 0,

"data_type": 0,

"dt_remap_to_type": 0,

"hsize": 1,

"vsize": 1,

"user_data_desc": 0,

1.2.1.13 subdrv_static_ctx 的成员解析

成员	意义
sensor_id	Sensor ID defined in kd_imgsensor.h.
reg_addr_sensor_id	Sensor register address where sensor ID is read. Up to 3 bytes.

成员	意义
i2c_addr_table	I2C write ID, end in 0xff, 4 for maximum. e.g., i2c_addr_table = {0x20, 0x6e, 0xff},.
eeeprom_info	The address of eeeprom_info_struct. If there is no eeeprom device, set to 0.
eeeprom_num	Array size of the eeeprom_info_struct. If there is no eeeprom device, set to 0.
resolution	Full pixel size of sensor output.
mirror	Set IMAGE_HV_MIRROR if the sensor orientation is 180 degrees difference from module orientation.
mclk	Overwrite MCLK frequency (MHz)
isp_driving_current	Overwrite MCLK driving current (mA)
sensor_interface_type	Data output interface type. Set default MIPI.
mipi_sensor_type	C-PHY or D-PHY
mipi_lane_num	How many lanes/trios of MIPI-PHY
ob_pedestal	OB offset. General is 64.
sensor_output_dataformat	Bayer order, 4-cell order and SW/HW remo.
ana_gain_def	Default analog gain. Set 4x. 1024base.
ana_gain_min	Minimum analog gain. 1024base.
ana_gain_max	Maximum analog gain. 1024base.
ana_gain_type	Sony: type 0; OV: type 1; Samsung: type 2; Hynix: type 3; GC: type 4
ana_gain_step	Minimum valid step of analog gain. 1024base.
ana_gain_table	Use valid analog gain table. Remove unsuitable value which is not linearity. 1024base.
ana_gain_table_size	Size of analog gain table
min_gain_iso	Set minimum ISO 100.
exposure_def	Default exposure line. Set 0x3D0.
exposure_min	Minimum exposure line
exposure_max	Maximum exposure line
exposure_step	Minimum valid step of exposure line
exposure_margin	Maximum margin of exposure line
frame_length_max	Maximum framelength
ae_effective_frame	AE effective frame
frame_time_delay_frame	The frame "frame length" setting takes effect. Sony sensor is filled in 3; other sensors are filled in 2.
start_exposure_offset	Parameter tuned in CTS sensor fusion test.
pdaf_type	Reference enum IMGSENSOR_PDAF_SUPPORT_TYPE_ENUM
hdr_type	Reference enum IMGSENSOR_HDR_SUPPORT_TYPE_ENUM

成员	意义
seamless_switch_support	If sensor supports seamless switch, set 1 to enable seamless switch function.
temperature_support	Set to 1 if sensor supports temperature sensor readout.
g_temp	Implement get temperature function.
g_gain2reg	Implement analog gain to register mapping rule.
s_gph	Implement set enable/disable group hold function.
s_cali	Implement the function to write data from eeprom_info_struct to sensor (e.g., QSC, cross-talk, ...)
Member	Meaning
reg_addr_stream	Sensor register address where streaming on/off is controlled.
reg_addr_mirror_flip	Sensor register address where mirror/flip on/off is controlled. No use to set to 0.
reg_addr_exposure	Sensor register address where exposure line is set. Up to 3 channels of exposure. Up to 3 bytes.
long_exposure_support	Set to 1 if sensor supports long exposure left shift function.
reg_addr_exposure_lshift	Sensor register address where long exposure left shift is set.
reg_addr_ana_gain	Sensor register address where analog gain is set. Up to 3 channels of exposure. Up to 3 bytes.
reg_addr_frame_length	Sensor register address where framelength is set.
reg_addr_temp_en	If "temperature_support" is set to 1. Sensor register address where temperature sensor on/off is controlled.
reg_addr_temp_read	If "temperature_support" is set to 1. Sensor register address where temperature output is read.
reg_addr_auto_extend	For Sony sensor, sensor register address where auto extend function enable/disable is controlled.
reg_addr_frame_count	For Samsung sensor, sensor register address where frame count is read to determine if sensor stream is off or not.
reg_addr_fast_mode	For Sony sensor, sensor register address where fast mode on/off is controlled.
init_setting_table	The address of sensor initial setting table
init_setting_len	Array size of the sensor initial setting table
mode	The address of subdrv_mode_struct
sensor_mode_num	Array size of the subdrv_mode_struct
list	The address of customized feature control list
list_len	Array size of the customized feature control list
checksum_value	The calculated value when Test Pattern output, for Camera Auto Test

1.2.1.14 配置 Sensor Mode

Subdrv_mode_struct mode_struct[] 这个结构体数组是用来存不同 Sensor Mode 的数据配置的。

下面以 Preview 为例子说明了一些需要客户修改的字段，对于 YUV sensor，一般后面的 Sensor Mode 和 Preview 是完全一样的。对于 raw sensor 可能会有差异，根据实际需要来配置。

mode_struct

hdr_mode:
0

raw_cnt:
1

exp_cnt:
1

pcik:

linelength:

framelength:

imgsensor_winsize_info

* full_w:

* full_h:

* x0_offset:

* y0_offset:

* w0_size:

sensor_output_dataformat:

SENSOR_OUTPUT_FORMAT_UYVY

```
61 }
62 ],
63 "mode_struct": [
64 {
65   "hdr_mode": 0,
66   "raw_cnt": 1,
67   "exp_cnt": 1,
68   "pcik": undefined,
69   "linelength": undefined,
70   "framelength": undefined,
71   "max_framerate": undefined,
72   "mipi_pixel_rate": undefined,
73   "readout_length": 0,
74   "read_margin": 10,
75   "framelength_step": 2,
76   "coarse_integ_step": 2,
77   "pdaf_cap": 1,
78   "ae_binning_ratio": 1465,
79   "fine_integ_line": 826,
80   "delay_frame": 3,
81   "dpc_enabled": 1,
82   "csi_param": {
83     "dphy_trail": undefined,
84     "dphy_data_settle": undefined,
85     "dphy_clk_settle": undefined,
86     "cphy_settle": undefined,
87     "legacy_phy": undefined,
88     "not fixed trail settle": undefin
94 },
95 "imgsensor_winsize_info": {
96   "full_w": undefined,
97   "full_h": undefined,
98   "x0_offset": undefined,
99   "y0_offset": undefined,
100   "w0_size": undefined,
101   "h0_size": undefined,
102   "scale_w": undefined,
103   "scale_h": undefined,
104   "x1_offset": undefined,
105   "y1_offset": undefined,
106   "w1_size": undefined,
107   "h1_size": undefined,
108   "x2_tg_offset": undefined,
109   "y2_tg_offset": undefined,
110   "w2_tg_size": undefined,
111   "h2_tg_size": undefined
112 },
113 "sensor_output_dataformat": 4,
114 "ana_gain_min": 1024,
115 "ana_gain_max": 65536,
116 "dig_gain_min": 1024,
117 "dig_gain_max": 65536,
118 "h2_tg_size": undefined
119 },
120 "sensor_output_dataformat": 4,
121 "ana_gain_min": 1024,
122 "ana_gain_max": 65536,
123 "dig_gain_min": 1024,
```

csi_param

dphy_trail:

dphy_data_settle:

dphy_clk_settle:

cphy_settle:

```

82 "csi_param": {
83   "dphy_trail": undefined,
84   "dphy_data_settle": undefined,
85   "dphy_clk_settle": undefined,
86   "cphy_settle": undefined,
87   "legacy_phy": undefined,
88   "not_fixed_trail_settle": undefined,
89   "dphy_csi2_resync_dmy_cycle": undefined,
90   "not_fixed_dphy_settle": undefined,
91   "dphy_init_deskew_support": undefined,
92   "clk_lane_no_initial_flow": 0,
93   "initial_skew": undefined
94 },
95 "imgsensor_winsize_info": {
96   "full_w": undefined,
97   "full_h": undefined,
98   "x0_offset": undefined,
99   "y0_offset": undefined

```

```

310 static struct subdrv_mode_struct mode_struct[] = {
311 {
312   .frame_desc = frame_desc_prev,
313   .num_entries = ARRAY_SIZE(frame_desc_prev),
314   .mode_setting_table = PARAM_UNDEFINED,
315   .mode_setting_len = PARAM_UNDEFINED,
316   .seamless_switch_group = PARAM_UNDEFINED,
317   .seamless_switch_mode_setting_table = PARAM_UNDEFINED,
318   .seamless_switch_mode_setting_len = PARAM_UNDEFINED,
319   .hdr_mode = HDR_NONE,
320   .raw_cnt = 1,
321   .exp_cnt = 1,
322   .pclk = 205100000,
323   .linelength = 5860,
324   .framelength = 1400,
325   .max_framerate = 300,
326   .mipi_pixel_rate = 200000000,
327   .readout_length = 0,
328   .read_margin = 10,
329   .framelength_step = 2,
330   .coarse_integ_step = 2,
331   .sensor_output_dataformat = SENSOR_OUTPUT_FORMAT_UYVY,
332   .multi_exposure_shutter_range[IMGSSENSOR_EXPOSURE_LE].min = 8,
333   .imgsensor_winsize_info = {
334     .full_w = 1280,
335     .full_h = 720,
336     .x0_offset = 0,
337     .y0_offset = 0,
338     .w0_size = 1280,
339     .h0_size = 720,
340     .scale_w = 1280,
341     .scale_h = 720,
342     .x1_offset = 0,
343     .y1_offset = 0,
344     .w1_size = 1280,
345     .h1_size = 720,
346     .x2_tg_offset = 0,
347     .y2_tg_offset = 0,
348     .w2_tg_size = 1280,
349     .h2_tg_size = 720,
350   },
351   .pdaf_cap = TRUE,
352   .imgsensor_pd_info = NULL,
353   .ae_binning_ratio = 1465,
354   .fine_integ_line = 826,
355   .delay_frame = 3,
356   .csi_param = {
357     .legacy_phy = 0,
358     .not_fixed_trail_settle = 1,
359     .not_fixed_dphy_settle = 1,
360     .dphy_trail = 0x18,
361     .dphy_data_settle = 0x27,
362     .cphy_settle = 0x13,
363     .clk_lane_no_initial_flow = 0,
364   },
365   .dpc_enabled = true, /* reg 0xb06 */
366 }, /* preview */

```

pclk, linelength, framelength
framerate, mipi pixel rate在这里配置

format 在这里配置

winsize info在这里配置

hs trail & settle在这里配置

1.2.1.15 驱动功能

1.2.1.15.1 Driver 操作函数 List

Driver 函数的 list 如下:

```
1073 static struct subdrv_ops ops = {
1074     .get_id = get_imgsensor_id,
1075     .init_ctx = init_ctx,
1076     .open = open,
1077     .get_info = common_get_info,
1078     .get_resolution = common_get_resolution,
1079     .control = common_control,
1080     .feature_control = common_feature_control,
1081     .close = common_close,
1082     .get_frame_desc = common_get_frame_desc,
1083     .get_temp = common_get_temp,
1084     .get_csi_param = common_get_csi_param,
1085     .update_sof_cnt = common_update_sof_cnt,
1086     .parse_ebd_line = common_parse_ebd_line,
1087     .get_sensor_usage = get_sensor_usage,
1088 };
```

在 adapter 名字开头的 imgsensordr 的代码里往往是通过 subdrv_call 的方式来调用到 driver 代码里的，就是上面的函数 list，比如：

```
ret = subdrv_call(ctx, get_id, &sensor_id);
```

上面这个调用就会 call 到上面函数 list 里的 get_sensor_id。

上面 list 里有很多 common 开头的函数，这个部分用的是通用的处理流程。不需要在 sensor driver 里特别实现，下面就不做过多介绍了。

针对贵司可能改到的 get_imgsensor_id 和 open 函数，请参考下面两个章节。

1.2.1.15.2 get_imgsensor_id 函数

开机 search sensor 时会通过 get_imgsensor_id 函数读取 ID。若能成功读到 ID，在 UI 上显示 Camera APP 的图标。

下面函数主要有两个功能：一获取 sensor ID，二判断是否有连接到模组。

```

751 static int get_imgsensor_id(struct subdrv_ctx *ctx, u32 *sensor_id)
752 {
753     // u32 deser_id, ret = 0;
754     // dts_id represents sensor 0/1/2/3 index
755     u32 compatible_dts_mask = 0xFFFFFFFF;
756     struct autocom_sector_data *sec_data = NULL;
757     struct adaptor_ctx *adp_ctx = container_of(ctx, struct adaptor_ctx, subctx);
758     sec_data = ctx->s_sec_data;
759     if (!sec_data) {
760         LOG_ERR("sec_data is NULL, pls check!");
761         return ERROR_DRIVER_INIT_FAIL;
762     }
763
764     compatible_dts_mask = sec_data->autocam_module_inf->compatible_dts_mask;
765     if ((compatible_dts_mask & (1 << adp_ctx->dts_idx)) == 0) {
766         LOG_ERR("firmware[%s] is not compatible with DTS sensor[%d]",
767             sec_data->file_name, adp_ctx->dts_idx);
768         return -ERROR_SENSOR_CONNECT_FAIL;
769     }
770     // ret = get_deserializer_id(ctx, &deser_id);
771     // if (ret < 0) {
772     //     LOG_ERR("SensorDevIdx: %d, can not find deserializer: %s", 1 << adp_ctx->dts_idx, sec_data->autocam_serdes_inf->deser_inf->name);
773     //     return ret;
774     // }
775     LOG_INF("SensorDevIdx: %d, expect cam module_id: 0x%x", 1 << adp_ctx->dts_idx, sec_data->autocam_module_inf->module_id);
776     get_cam_module_id(ctx, sensor_id);
777     // if sensor_id read by i2c is not the same as the one from firmware data, return connect failure.
778     if (*sensor_id != sec_data->autocam_module_inf->module_id) {
779         *sensor_id = 0xFFFFFFFF;
780         return -ERROR_SENSOR_CONNECT_FAIL;
781     }
782     LOG_INF("sensor_id = 0x%x, matched with expect module_id", *sensor_id);
783     return 0;
784 }

```

1.2.1.15.3 open 函数

每次进 camera 时会调用。

读 sensor_id, 确认 I2C 通信是否正常。

调用 sensor_init 函数初始化 ctx 结构体中的一些变量。

```

int open(struct subdrv_ctx *ctx)
{
    u8 retry = 2;
    u32 sensor_id = 0;
    u32 scenario_id = 0;

    LOG_INF("%s:\n", __func__);
    get_imgsensor_id(ctx, &sensor_id);
    autocam_init(ctx);
    retry = 3;
    do {
        if (Check_Sensor_Signal(ctx)) {
            LOG_ERR("Sensor Not Connect");
            mdelay(5);
            retry--;
        } else
            break;
    } while (retry > 0);

    if (retry == 0) {
        LOG_ERR("OPEN FAIL!Sensor Not Connect");
        return -ERROR_SENSOR_CONNECT_FAIL;
    }

    /* move camera init to initctx(), void max9286 init too much time */

    spin_lock(&autocam_drv_lock);
    ctx->current_scenario_id = scenario_id;
    ctx->pclk = ctx->s_ctx.mode[scenario_id].pclk;
    ctx->line_length = ctx->s_ctx.mode[scenario_id].linelength;
    ctx->frame_length = ctx->s_ctx.mode[scenario_id].framelength;
    ctx->current_fps = ctx->pclk / ctx->line_length * 10 / ctx->frame_length;
    ctx->readout_length = ctx->s_ctx.mode[scenario_id].readout_length;
    ctx->read_margin = ctx->s_ctx.mode[scenario_id].read_margin;
    ctx->min_frame_length = ctx->frame_length;
    ctx->autoflicker_en = FALSE;
    ctx->test_pattern = 0;
    ctx->ihdr_mode = 0;
    ctx->pdaf_mode = 0;
    ctx->hdr_mode = 0;
    ctx->extend_frame_length_en = 0;
    ctx->fast_mode_on = 0;
    ctx->sof_cnt = 0;
    ctx->ref_sof_cnt = 0;
    ctx->is_streaming = 0;
    spin_unlock(&autocam_drv_lock);

    return ERROR_NONE;
}

```

注:**1. 关于分辨率的说明**

由于场景不同，给到 User Space 的图的大小和实际 seninf 收取的 VC 的大小不同，所以在 driver 里会有两个分辨率，客户可以以 max96712isx 这个非组帧 AVM 的 driver 为例。搜索下面的宏可以对照配置。

```

16 #include "kd_imgsensor_define_v412.h"
17 #include "kd_imgsensor_errcode.h"
18
19 #include "adaptor-subdrv.h"
20 #include "adaptor-i2c.h"
21
22 #include "max96712isxmiyuv_Sensor.h"
23
24 #define PFX "max96712isx_mipi_yuv_camera_sensor"
25
26 #define VC 0
27
28 #define SUPERFRAME_WIDTH 5120
29 #define SUPERFRAME_HEIGHT 720
30
31 #define FRAME_WIDTH 1280
32 #define FRAME_HEIGHT 720
33
34 #define MAX96712ISX_ID_REG 0x00U
35 #define MAX96712ISX_SENSOR_ID 0x54
36
37 #define read_cmos_sensor(...) subdrv_i2c_rd_u8(__VA_ARGS__)
38 #define write_cmos_sensor(...) subdrv_i2c_wr_u8(__VA_ARGS__)
39
40 static int vsync_notify(struct subdrv_ctx *ctx, unsigned int sof_cnt);
41 /* STRUCT */
42
43 static struct subdrv_feature_control feature_control_list[] = {
44 };

```

这个是上层看到的大图的分辨率，比如非组帧avm，这里的width会是5120

这个是单个vc 的分辨率

2. 关于 dts 里 GPIO 供电的部分，还可以选择 regulator 封装的方式

用法如下：

```

109 > gpio46_regulator: gpio-46 {
110 >     compatible = "regulator-fixed";
111 >     regulator-name = "gpio46_regulator";
112 >     regulator-min-microvolt = <1800000>;
113 >     regulator-max-microvolt = <1800000>;
114 >     gpio = <gpio 46 0x0>;
115 >     enable-active-high;
116 >     status = "okay";
117 > };

```

```

686 > sensor0: sensor0@52 {
687 >     compatible = "mediatek,ingsensor";
688 >     sensor-names = "max96712isx_mipi_yuv";
689 >     reg = <0x52>;
690
691 >     #thermal-sensor-cells = <0>;
692 >     pdn-supply = <gpio46_regulator>;
693
694 >     clocks = <&vlp_cksys_clk CLK_CK_UNIVPLL_192M_D32>,
695 >     <&vlp_cksys_clk CLK_CK_UNIVPLL_192M_D16>,
696 >     <&vlp_cksys_clk CLK_CK_F26M_CK_D2>,
697 >     <&vlp_cksys_clk CLK_CK_UNIVPLL_192M_D10>,
698 >     <&vlp_cksys_clk CLK_CK_UNIVPLL_192M_D8>,
699 >     <&vlp_cksys_clk CLK_CK_UNIVPLL_D6_D16>,
700 >     <&vlp_cksys_clk CLK_CK_UNIVPLL_D6_D8>,
701 >     <&vlp_cksys_clk CLK_VLP_CK_CAMT08_SEL>;
702 >     clock-names = "6", "12", "13", "19.2", "24", "26", "52", "mclk";
703
704 >     status = "okay";
705
706 >     port {
707 >     >         sensor0_out: endpoint {
708 >             remote-endpoint = <&seninf_csi_port_4_in>;
709 >         };
710 >     };
711 > };

```

3. UT 测试指令:

```
C:\Users>adb shell sentest_v4l2 //just search sensor, not streaming
```

```
[show_Sensors]sensorNum 2
[show_Sensors]name:SENSOR_DRVNAME_OV05A20_MIPI_RAW type:0
[show_Sensors]index:0, SensorDevIdx:1
[show_Sensors]name:SENSOR_DRVNAME_IMX576_MIPI_RAW type:0
[show_Sensors]index:1, SensorDevIdx:2
[main]Param: 1 <sensorDev> <scenario> <fps>
[main]<sensorDev> : main(1), Sub(2), Main2(4), sub2(8), Main3(16)
[main]<scenario> : Pre(0), Cap(1), VD(2), slim1(3), slim2(4)
```

显示 sensorNum 不为 0，并且展示了传感器的具体名称，说明传感器搜索成功。接下来测试传感器的输出

```
C:\Users> adb shell sentest_v4l2 1 0 //search sensor, then open main (1) sensor
preview (0)
```

```
[test_SensorInterface]name:SENSOR_DRVNAME_MAX96712ISX_MIPI_YUV type:1
[test_SensorInterface]index:0, SensorDevIdx:1
[test_SensorInterface]scenario(0) , binning type = 1465
[test_SensorInterface]scenario(0) , rawFmtType = 0
[test_SensorInterface]scenario(0) , OB_pedestal = 64
[test_SensorInterface]scenario(0) , sensorFormatOrder = 4
[test_SensorInterface]scenario(0) , seamless_targets_count = 0
[test_SensorInterface]Sensor index:0, scenarioId:0
[test_SensorInterface]crop w:1280, h:720
[test_SensorInterface]HDR_Support:0
[test_SensorInterface]scenario(0) , RGBW_Support = 0
[test_SensorInterface]scenario(0) , PDAF_Support = 0, dpc(1), pdc(0)
[test_SensorInterface]scenario(0) , Def_MirrorFlip = 3
[test_SensorInterface]scenario(0) , sensorFormatCellType = 0
[test_SensorInterface]SENSOR_CMD_GET_SENSOR_CROP_WIN_INFO ret 0
[test_SensorInterface]SENSOR_CMD_GET_SENSOR_RGBW_OUTPUT_MODE rgbw mode = 0
[test_SensorInterface]ana gain min/max = 1024/65536
[test_SensorInterface]dig gain min/max = 1024/16384
[test_SensorInterface]dig gain step = 4
[test_SensorInterface]SENSOR_CMD_GET_OFFSET_TO_START_OF_EXPOSURE exposure_offset = 500000
[test_SensorInterface]pHalSensor->powerOn_start
[test_CID][test g volatile ctrl] scenario (by set format)
[test_CID][V4L2_CID_VBLANK] ret:0 VB:630
[test_CID][V4L2_CID_HBLANK] ret:0 HB:4434
[test_CID][V4L2_CID_MTK_SENSOR_PIXEL_RATE] ret:0 pixel rate:200000000
[test_CID][V4L2_CID_MTK_STAGGER_INFO] ret:0 count:0 order:
[test_CID][V4L2_CID_MAX_EXP_TIME]ret 0 exposure 0, max_exp_time 0
[test_CID][V4L2_CID_MAX_EXP_TIME]ret 0 exposure 1, max_exp_time 0
[test_CID][V4L2_CID_MAX_EXP_TIME]ret 0 exposure 2, max_exp_time 0
[streamon]sensor: idx=0 devpath=/dev/v4l-subdev26 drvname=max96712isx_mipi_yuv
[streamon]seninf: devpath=/dev/v4l-subdev4 cid=0x98f901 en=1
[test_SensorInterface]main/rolling_shutter=20571428
[test_SensorInterface]main/exp_margin=1371
[test_SensorInterface]main/atv-Continuous=1
[test_SensorInterface]main/atv-BypassSensorScenario=0
[test_SensorInterface]main/min_shutter=24 shutter=2
[test_SensorInterface]TgInfo[0] = 0
[test_SensorInterface]pixelMode[0] = 0
[test_SensorInterface]mipi rate = 200000000
[test_SensorInterface]wait enter key to end sensorIndex:0...
```

4. 查看 seninf 状态的指令如下，查到结果后，可以发给联发科技确认

```
255]auto8676p1_64_bsp:/ # cat /sys/devices/platform/soc/1a00e000.seninf-top/status
clk-fmeter-isp(687576) clk-fmeter-csi0(26000) clk-fmeter-csi1(0) clk-fmeter-csi2(0) clk-fmeter-csi3(312000) clk-fmeter-csi4(0) clk-fmeter-csi5(0)

[seninf-8] port 3 intf 6 test 0 cphy 0 lanes 0
[seninf-7] port 3 intf 6 test 0 cphy 0 lanes 0
source sensor7 flags 0x4

[seninf-6] port 3 intf 6 test 0 cphy 0 lanes 4
source sensor6 flags 0x5
csi2_line_frame_num 0x00000000, seninf_irq_stat 0x00000000
csi2_packet_status 0x12c0012b
csi2_packet_cnt_status 0x000004fb
rx-ana settle ck 0x00 dt 0x1a
rx-ana trail en 0 param 0x00
CSI2X_MAC_CSI2_RESYNC_MERGE_CTRL 0x00000106 CDPHY_RX_ANA_SETTING_1 0x04340244 DPHY_RX_SPARE0 0x000000f1
data not enough cnt : <0>
err_lane_resync_cnt : <0>
crc_err_cnt : <0>
ecc_err_double_cnt : <0>
ecc_err_corrected_cnt : <0>
after write clear csi_irq 0x0 seninf_irq_stat 0x00000000
[0] vc 0x0 dt 0x2b mux 1 cam 0
mux[1] en 1 src 6 irq_stat 0xc
fifo overrun_cnt : <0>
cam[0] en 1 Src 8 exp 0x8700f00 res 0x8700f00 err 0x0 irq_stat 0x100
size_err_cnt : <0>
--- mux meter ---
width 3840 height 2160
h_valid 0, h_blank 0
v_valid 0, v_blank 0
mipi_pixel_rate 0
v_blank 0 us
h_blank 0 us
line_time 0 us

[seninf-2] port 2 intf 4 test 0 cphy 0 lanes 0
[seninf-1] port 1 intf 2 test 0 cphy 0 lanes 0
auto8676p1_64_bsp:/ #
```

5. 模组上电情况下，通过 I2C 指令直接访问 max96712

下面是读 0x108 reg 的值：

```
echo 0x52 0x108 > /sys/devices/platform/soc/13b30000.i2c/i2c-8/8-0052/debug_i2c_ops
```

```
D:\Debug>adb shell
auto8676p1_64_bsp:/ # echo 0x52 0x108 > /sys/devices/platform/11cc0000.i2c/i2c-8/8-0052/debug_i2c_ops
auto8676p1_64_bsp:/ # cat /sys/devices/platform/11cc0000.i2c/i2c-8/8-0052/debug_i2c_ops
max96712_mipi_yuv i2c read 0x00000108 = 0x00000062
auto8676p1_64_bsp:/ #
```

下面是写 0x8a0 reg 为 0x84：

```
echo 0x52 0x8a0 0x84> /sys/devices/platform/11cc0000.i2c/i2c-8/8-0052/debug_i2c_ops
```

6. 开 sensor driver log 的指令

```
echo 1 > /sys/module/imgsensor/parameters/sensor_debug
```

7. 假如串口 log 打印停止了的话，开 UART 的指令

```
adb shell setprop persist.vendor.uartconsole.enable 1
```

1.2.2 如何添加一个新的 Tuning 文件

设计原理参考：MT8678 JSON Design CN.pptx 文档

添加方法:

Porting 一个新 sensor, 在 tuning 的部分, 只需要新增一份 `${SENSOR_DRIVER_NAME}_tuning_param.json` 文件即可。

注:

- 若需编译进 image, 则将 JSON 文件放到 `\vendor\mediatek\proprietary\hardware\mtkcam-core\external\firmware\tuning\` 目录
- 若需单独额外增加 sensor 的 tuning JSON file, 则使用 adb push tuning JSON file 到 `vendor/firmware/camera/tuning/` 目录

`${SENSOR_DRIVER_NAME}_tuning_param.json` 这个文件可以复制一份现有模板, 改一下 `sensor_driver_name` 的名字即可。

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。